

University of Rochester
Warner School of Education

Innovative Unit Plan
Inclusive Computer Science Unit Planning
for Neurodivergent & CLD Learners

**Using Placement Evidence to Separate Access Barriers from
Computational Thinking**

**Minecraft Education Coding FUNdamentals as an Equity-Centered CS
Context**

Piter Garcia
Teaching & Curriculum Graduate Student

March 19, 2026

ED452B — Instructional Strategies for Inclusive Classrooms B
Spring 2026 · Instructor: Elyse Schirmer

Design Statement: This innovative unit plan adapts the Minecraft Education: Coding FUNdamentals (Upper Primary Coding Fundamentals) sequence into an inclusive elementary computer science unit grounded in MTSS, UDL, and culturally and linguistically sustaining design. The central planning commitment is to use placement evidence to separate access barriers from computational thinking, especially for neurodivergent and culturally and linguistically diverse learners. All design decisions are directly informed by placement observations at Pine Brook Elementary (March 2026).

Contents

1	☰ Warner Required Lesson Plan — Header	2
2	☰ Introduction	2
3	🏠 Unit Context & Rationale	3
3.1	Unit Context	3
3.2	Community, School, and Classroom Characteristics	3
3.3	Broader Curricular Context	3
3.4	Rationale for Unit Design	3
3.5	Three Target Students	5
4	🧠 Theoretical Framework	6
4.1	1. Karten’s Inclusion Model & MTSS	6
4.2	2. Universal Design for Learning (UDL)	6
4.3	3. Culturally Sustaining Pedagogy & CS Identity	6
5	🎯 Long-Term Goals & Standards	7
5.1	Unit Goal	7
5.2	Essential Question	7
5.3	Big Ideas	7
5.4	Unit Objectives	7
5.5	NYS & CSTA Standards	8
6	📊 Assessment	8
6.1	Assessment Plan by Lesson	9
7	📚 Pedagogy & Implementation	9
7.1	Instructional Approach	9
7.2	Core Teaching Strategies	9
7.3	Tier 1 Universal Supports (whole class)	10
7.4	Tier 2 Targeted Scaffolds (identified students)	10
8	📅 Lesson Sequence Overview	11
9	✍ Detailed Lesson Plans — Warner Procedure Tables	11
9.1	Lesson 1: Basic Moves	12
9.2	Lesson 2: Sequencing to Reach a Goal	14
9.3	Lesson 3: Debugging & Revision	16
9.4	Lesson 4: Efficient Solutions & Creative Build	18
10	💬 Reflection Prompts (Post-Implementation)	20
11	💡 Unit Analysis	21
11.1	Analysis of Student Learning	21
11.2	Observed Learning Patterns	21
11.3	Unit Strengths	22
11.4	Problems, Dilemmas, and Next Revision	22

11.5 Target Student Assessment and Future Instructional Decisions	23
11.6 Collaboration	23
12  References (APA 7th Edition)	24

1 ☰ Warner Required Lesson Plan — Header

CANDIDATE	DATE	COOPERATING TEACHER
Piter Garcia pgarcia8@u.rochester.edu	March 19, 2026	Mr. Derek Romig derek.romig@greececsd.org

GRADE LEVEL	SUBJECT / PROGRAM	DURATION
Grade 4	IGNITE — Computer Science & Digital Literacy, Pine Brook Elementary, Greece CSD	4 sessions × 45 min Sessions 31–34 of 35

LESSON TITLE	Inclusive Computer Science Unit Planning for Neurodivergent and CLD Learners: Minecraft Education, Access Barriers, and Computational Thinking
--------------	--

2 ☰ Introduction

This innovative unit plan is titled *Inclusive Computer Science Unit Planning for Neurodivergent and CLD Learners: Using Placement Evidence to Separate Access Barriers from Computational Thinking*. It is built around *Minecraft Education: Coding FUNdamentals*, specifically the Upper Primary Coding Fundamentals sequence (Lessons 1–4). The unit is designed for an inclusive Grade 4 IGNITE computer science rotation at Pine Brook Elementary and spans four 45-minute lessons. The purpose of the unit is to teach foundational computational thinking through a high-interest digital environment while making the learning sequence accessible to neurodivergent and culturally and linguistically diverse learners.

The unit is innovative in two ways. First, it uses a game-based coding environment in which students program an Agent avatar using block code (MakeCode), producing immediate visual feedback on whether their algorithm is correct. Second, it redesigns that environment through inclusive instructional structures so that access to the lesson is not limited by reading load, startup difficulty, executive-function demands, language demands, or prior familiarity with the platform — challenges that were directly observed during the March 2026 Minecraft pilot lesson at Pine Brook. The central analytic move of the revision is to treat student difficulty as evidence to interpret carefully: a student who cannot enter the correct world, read the NPC instructions, manage the interface, or explain in writing may still be developing computational thinking. Therefore, the unit separates access data from coding data before making claims about learning.

3 Unit Context & Rationale

3.1 Unit Context

Candidate	Piter Garcia
School / Setting	Pine Brook Elementary, IGNITE Computer Science & Digital Literacy rotation
Grade Band	Grade 4
Content Area	Computer Science / Digital Literacy
Unit Title	Inclusive Computer Science Unit Planning for Neurodivergent and CLD Learners: Using Placement Evidence to Separate Access Barriers from Computational Thinking
Unit Length	4 lessons, approximately 45 minutes each (Sessions 31–34 of 35-session IGNITE year, May–June 2026)
Instructional Setting	Rotating inclusive classes; mixed readiness, mixed digital fluency, varied support needs

3.2 Community, School, and Classroom Characteristics

Pine Brook’s IGNITE program serves elementary learners through short rotation-based computer science instruction. Students enter the classroom with uneven prior experiences in coding, digital navigation, keyboard use, reading stamina, and collaborative work habits. The March 2026 placement pilot revealed that students vary in how independently they can follow multi-step digital directions, persist through troubleshooting, and ask for help appropriately. The classroom context also includes neurodivergent learners and culturally and linguistically diverse (CLD) learners whose competence may not be visible when a lesson depends heavily on fast reading, long verbal directions, or self-managed transitions.

3.3 Broader Curricular Context

This unit sits at the capstone of the IGNITE 35-session rotation (Sessions 31–35), positioned after students have worked through Book Creator, GIF animation, Chromebook camera, and Tinkercad. Students bring comfort with Google Classroom workflows and basic keyboard use. The Minecraft coding unit introduces block-based programming (MakeCode) and agent-based algorithm execution for the first time — concepts that transfer to robotics, Code.org, and other block environments used in IGNITE. The primary standard is NYS 4-6.CT.1 (create, test, and revise algorithms using block coding), supported by CSTA 1A-AP-08, -10, -11, -14 and ISTE Student Standard 5c.

3.4 Rationale for Unit Design

The March 2026 pilot lessons conducted by Mr. Romig and co-supported during placement provided direct design intelligence. Across the March 13, March 17, March 23, and March 26 placement transcripts, several patterns repeated: students needed an explicit click-path into the lesson, many became stuck when NPC directions were skipped or skimmed, the Agent’s location and orientation had to be made visible before coding made sense, and the timed dual-plate challenge required students to understand the relationship between the student’s body and the Agent’s programmed movement. These were not small management details; they were access variables that affected

whether computational thinking could be seen.

The transcript evidence also showed that students' Minecraft expertise was uneven. Some students moved quickly, helped peers, or understood game mechanics before the formal coding objective was secure. Others needed repeated prompting to enter the correct room, use the touchpad rather than the touchscreen, rename the chat command, type the command correctly, count blocks, or remember that the Agent could move through air or water. Because of this, the unit design avoids interpreting speed, reading compliance, or interface independence as the same thing as coding competence. Instead, it treats access, algorithm design, debugging, explanation, and efficiency as related but separate evidence streams.

- › Whole-class launch routine for the exact click-path into the lesson (Play > Library > Computer Science > Block Coding > Upper Primary Coding Fundamentals > Lesson 1)
- › Explicit NPC read-aloud routine before any coding begins — teacher models this every session
- › Structured peer support through Driver / Navigator roles (removes ambiguity that escalates Students B and C)
- › “Stop, Read, Predict” protocol before every Run — builds prediction habit, catches EF impulsivity (Student A)
- › Spatial vocabulary pre-teaching before coordinate commands appear in Lesson 2
- › Extension pathways (Expert Role, loop challenge, Build Bridges world) to retain high-performers (Kade, Hadi, Hani, Lorenzo)
- › Stuck protocol posted on wall and on desk cards — normalizes iteration, reduces regulation breaks
- › Assessment that distinguishes interface/startup difficulty from conceptual coding difficulty

- › **March 13 Minecraft Coding Fundamentals:** students needed repeated modeling for the entry screen, yellow-wall room, Agent setup, and command execution; this supports the click-path launch routine and visual setup checklist.
- › **March 17 Agent Puzzles:** the timed gold-plate challenge showed that students needed to write one complete code sequence, coordinate their own movement with the Agent's movement, and understand that the Agent can float or travel underwater; this supports explicit success criteria and strategy comparison.
- › **March 23 Rubric / lesson design meeting:** feedback emphasized that rubric language needed to specify observable evidence, not vague terms such as “understood”; this supports the revised assessment categories and evidence table.
- › **March 26 coding workshop:** peer expertise and table-based help emerged naturally, especially when some students had already moved ahead; this supports Driver/Navigator roles and Expert Helper routines that channel peer help without letting one student take over.

3.5 Three Target Students

Target Student	Profile & Likely Barrier	Planned Supports
Student A (Executive Function)	Neurodivergent learner; strong verbal participation and creative ideas but acts quickly without reading sequential directions. In the pilot session, this student skipped NPC text, clicked randomly, and became frustrated when the Agent did not behave as expected — consistent with an executive-function profile (impulsivity, difficulty with multi-step tasks in novel digital environments).	› Chunked task cards (3 steps only per card) › “Stop, Read, Predict” enforced at every Run › Visible desk checklist › Teacher checkpoint before first code execution › Reduced decision-load at each phase
Student B (CLD / Language)	CLD learner; understands concepts more readily than dense NPC text reflects. Fast whole-group verbal directions and English-only supports mask competence. In the pilot, this student engaged well with visual demonstrations and peer explanation, suggesting oral and visual channels are stronger access routes than text.	› Bilingual vocabulary anchor chart › Visual NPC transcript card (text pre-printed) › Partner read-aloud of instructions › Oral explanation of code as valid assessment evidence › Sentence frames on exit tickets › Immersive Reader enabled in Minecraft settings
Student C (Social-Communication / Regulation)	Neurodivergent learner with social-communication and sensory regulation needs. Engages deeply with structured tasks but becomes overwhelmed in open-ended or loud digital environments. Ambiguous collaboration expectations increase anxiety. Thrives when routine is predictable and transitions are explicitly signaled.	› Explicit pre-assigned Driver/Navigator roles › Posted session agenda with time markers › Quiet signal system (“stuck card” on desk) › First-choice seat near teacher › Structured exit ticket with sentence starters › Identical exit ticket format across all 4 lessons

4 Theoretical Framework

This unit is grounded in three complementary theoretical lenses that together define an inclusive instructional design stance: Karten’s Inclusion Model and MTSS, Universal Design for Learning (UDL), and Culturally Sustaining Pedagogy as it applies to technology access and CS identity formation.

4.1 1. Karten’s Inclusion Model & MTSS

Karten (2021) frames inclusive instruction not as a set of accommodations added to a standard lesson, but as a systemic design orientation: supports are built into the infrastructure of learning so that all students can access grade-level content. Multi-Tiered Systems of Support (MTSS) operationalizes this through Tier 1 universal design (available to all), Tier 2 targeted scaffolds (for students showing specific access barriers), and Tier 3 intensive supports (beyond this unit’s scope). This unit applies the MTSS lens by front-loading Tier 1 supports — visual vocabulary, consistent debug protocols, multimodal directions, predictable lesson structure — rather than waiting for students to fail and then adding supports reactively. The pilot evidence supports this choice directly: students with executive-function and language processing needs are often the first to become stuck in novel digital environments, and their difficulty is frequently misread as disengagement or low ability.

4.2 2. Universal Design for Learning (UDL)

The UDL framework (CAST, 2018) provides three principles guiding this unit. **Multiple Means of Representation:** students access algorithm concepts through physical/unplugged modeling, NPC text, verbal teacher modeling, and visual vocabulary charts. **Multiple Means of Action and Expression:** students demonstrate computational thinking through MakeCode sequences, planning sheets, oral partner explanations, exit tickets, and visual documentation — not text alone. **Multiple Means of Engagement:** choice within tasks, creative extension builds (loop challenge, Build Bridges world), and a low-stakes debugging culture that normalizes iteration rather than penalizing errors. These principles directly address findings from the pilot: students who struggled with coordinate commands needed spatial vocabulary pre-teaching (representation); students whose competence was masked by English-only interfaces needed oral explanation options (expression); students who disengaged when stuck needed structured extension pathways and stuck protocols (engagement).

4.3 3. Culturally Sustaining Pedagogy & CS Identity

Paris and Alim’s (2017) culturally sustaining pedagogy calls for instruction that affirms and extends students’ existing cultural and linguistic assets. In the context of Minecraft, this means treating students’ prior game knowledge as expertise rather than distraction. In the March 2026 pilot, Mr. Romig’s pedagogical stance modeled this directly: students who already knew advanced Minecraft mechanics (Kade, Hadi, Hani who had crossed the hard navigation section; Lorenzo who did screen capture) were positioned as peer resources rather than outliers to manage. The unit formalizes this through the Expert Role structure in Lessons 3–4. For CLD students particularly, Minecraft is a globally prevalent game that crosses language barriers, and leveraging existing game proficiency builds the positive CS identity that literature identifies as a prerequisite for sustained engagement (Margolis et al., 2017).

- › **Karten / MTSS:** Tier 1 supports embedded from Lesson 1; debug protocol consistent across all 4 lessons; visual routine anchors each session. *Observed basis:* EF/regulation students stuck immediately when NPC text was skipped — support now built-in, not reactive.
- › **UDL:** Multiple representation modes (unplugged, visual, oral, digital); multiple expression forms (code, sketch, oral, written); engagement through choice and iteration culture. *Observed basis:* language processing students could not access dense NPC text; oral explanation now formally valid.
- › **Culturally Sustaining Pedagogy:** Prior game knowledge valued as expertise; Expert Role structure in Lessons 3–4; student creative choices respected within coding task. *Observed basis:* Kade, Hadi, Hani, Lorenzo as natural peer resources in pilot — now formalized for equity.

5 🎯 Long-Term Goals & Standards

5.1 Unit Goal

Students will develop foundational computational thinking by using block coding in Minecraft Education to create, test, debug, and explain algorithms in an inclusive, access-forward learning environment.

5.2 Essential Question

How can students use algorithms, debugging, and structured collaboration to solve coding problems in a digital environment — regardless of prior game experience, language background, or access needs?

5.3 Big Ideas

- › Algorithms are precise, ordered instructions — sequence matters.
- › Debugging is an expected and integral part of coding, not evidence of failure.
- › Planning, testing, and revising improve program quality.
- › Inclusive supports help more learners participate meaningfully in computer science.
- › Prior knowledge and game expertise are legitimate starting points for coding learning.

5.4 Unit Objectives

1. Students will create and test algorithms using Minecraft Education's block-coding tools (Make-Code Agent).
2. Students will use sequencing, debugging, and planning to improve code over multiple attempts.
3. Students will explain how their code works using CS vocabulary: algorithm, sequence, debug, agent, loop, coordinate.
4. Students will collaborate productively through structured Driver/Navigator peer roles.

5.5 NYS & CSTA Standards

Standard	Full Statement
NYS 4-6.CT.1	Develop a program that uses a sequence of commands to accomplish a task, modify the program to solve a related problem, and test the program to determine if it works as intended.
CSTA 1A-AP-08	Model daily processes by creating and following algorithms (sets of step-by-step instructions) to complete tasks.
CSTA 1A-AP-10	Develop programs with sequences and simple loops, to express ideas or address a problem.
CSTA 1A-AP-11	Decompose (break down) the steps needed to solve a problem into a precise sequence of instructions.
CSTA 1A-AP-14	Debug (identify and fix) errors in an algorithm or program that includes sequences and simple loops.
ISTE Student 5c	Break problems into component parts, extract key information, and develop descriptive models to understand complex systems or facilitate problem-solving.

6 Assessment

- Students are not assessed only by whether they finish the Minecraft task.
- Assessment includes planning quality, partner explanation, debugging strategy, and task completion — multiple evidence types.
- Teacher notes distinguish: procedural/interface difficulty vs. reading difficulty vs. conceptual difficulty.
- Exit tickets and conferences provide evidence for learners who cannot demonstrate understanding fully through the in-game task alone.
- Assessment avoids conflating access barriers (device issues, NPC language load, navigation) with coding competence.
- Transcript-informed evidence categories make the rubric more observable: startup independence, goal interpretation, whole-sequence planning, debugging moves, peer support, and explanation after completion.

6.1 Assessment Plan by Lesson

Lesson	Focus	Lesson Goal	Formative Assessment	Evidence Collected
1	Basic Moves	Navigate Minecraft Education, open MakeCode, use basic movement blocks to move the Agent.	Teacher observation of Minecraft startup; prediction before Run; quick conference during first code attempt; exit ticket 1	Startup independence, short sequence accuracy, basic error noticing
2	Sequencing	Plan and code a longer algorithm using Plan Before You Code worksheet.	Planning sheet review; observation of partner talk; teacher check-in before code execution	Plan quality, sequence accuracy, use of vocabulary
3	Debugging	Identify errors using the debug protocol and revise code; explain the fix.	Debug conference (“What did you expect? What happened?”); code revision check; peer explanation	Evidence of revision using protocol; explanation of bug and fix
4	Efficient Solutions & Reflection	Compare solution strategies using loops, complete loop challenge or creative build; unit reflection.	Completion conference; Unit Reflection Exit Ticket 4	Loop use, explanation of efficiency, final reflection

7 Pedagogy & Implementation

7.1 Instructional Approach

The unit uses explicit instruction, gradual release, structured collaboration, and reflective closure. Each lesson follows a stable five-part sequence: (1) warm-up / engaging opening, (2) teacher model, (3) guided practice, (4) independent or partner work, (5) closure / formative assessment. This consistent routine serves both instructional and inclusive purposes — students with regulation and EF needs thrive when structure is predictable, and the teacher can spend less time re-explaining procedure and more time supporting thinking.

7.2 Core Teaching Strategies

- › Think-aloud modeling during startup, NPC navigation, and MakeCode coding
- › Explicit teaching of vocabulary and controls before independent use
- › Driver / Navigator partner roles — defined, bounded, role-switched at midpoint
- › “Plan Before You Code” worksheet for Lessons 2 and 4 — separates planning from execution
- › Immediate feedback through prediction, run, debug cycle — algorithm testing loop
- › Short reflective exit ticket at lesson end — consistent format across all 4 lessons

7.3 Tier 1 Universal Supports (whole class)

Area of Need	Tier 1 Instructional Response
Startup and navigation difficulty	Click-path chart posted and narrated at launch; whole-class launch routine; teacher models lesson entry step by step every session
Reading load / NPC directions	Teacher reads NPC aloud as a class routine; NPC text printed on physical card; immersive reader pre-enabled in Minecraft settings
Executive-functioning needs	“Stop, Read, Predict” protocol before every Run; chunked task cards; visible desk checklist; teacher checkpoint before code execution
Social / regulation needs	Pre-assigned Driver/Navigator roles; posted session agenda with time markers; stuck card on desk for self-advocacy; predictable lesson structure across all 4 sessions
Language-processing needs	Simplified directions; bilingual vocabulary anchor chart; visual vocabulary support; oral explanation options; sentence frames on all exit tickets

7.4 Tier 2 Targeted Scaffolds (identified students)

Student Profile	Additional Scaffolds Beyond Tier 1
Student A (EF)	Required planning sheet before coding; 3-step task card (not 5); mandatory teacher check-in before first Run; proximity support during independent work
Student B (CLD / Language)	Pre-printed NPC transcript card; bilingual peer partner for reading aloud; oral or demonstrated code explanation accepted; translated vocabulary list
Student C (Regulation)	Assigned seat near teacher; option (a) assigned by teacher in Lesson 4 to reduce choice-overload; explicit multiplayer rules posted before joining shared world; quiet signal

8 Lesson Sequence Overview

#	Title	Main Content	Key Activities	Products
1	Basic Moves	Movement controls, Agent, MakeCode entry	Unplugged warm-up, teacher model, click-path launch, first short movement code, exit ticket	Short movement sequence + exit ticket 1
2	Sequencing to Reach a Goal	Longer ordered algorithms, planning	Compare-two-sequences warm-up, Plan Before You Code worksheet, Driver/Navigator partner coding	Planning sheet + working code + exit ticket 2
3	Debugging & Revision	Identifying and fixing errors using protocol	Buggy code warm-up, debug protocol mini-lesson, tiered debug challenge, Expert Role for finishers	Revised code + bug explanation + exit ticket 3
4	Efficient Solutions & Creative Build	Loops, comparing solutions, creative capstone	12 blocks vs. loop comparison, loop introduction, Build Bridges creative build, unit reflection	Loop-based build or code + Unit Reflection Exit Ticket 4

9 Detailed Lesson Plans — Warner Procedure Tables

Objective (applies across all 4 lessons): Students will create, test, and revise block-coded algorithms in Minecraft Education, demonstrating algorithmic sequencing and debugging through code, oral explanation, and written planning — using Tier 1 universal supports throughout and Tier 2 targeted scaffolds for identified students.

9.1 Lesson 1: Basic Moves

Lesson Goal	Students will navigate Minecraft Education, open MakeCode, and use basic movement blocks to move the Agent toward a goal.
Objectives	Use WASD + mouse + spacebar to move; open MakeCode with C; place a short movement sequence; notice when the Agent does not do what was expected.
Materials	Minecraft Education, Chromebook, click-path chart, controls anchor chart, directional warm-up cards, Exit Ticket 1.

0–5 min — Engaging Opening

Teacher says / does: Introduce the lesson as coding rather than building. Run an unplugged challenge with directional cards and a three-step movement sequence.

Students say / do: Physically follow directional commands, then generate one short sequence for the class to perform.

Differentiation / supports: A: printed sequence card for reference. B: icon-only directional card. C: bounded opening role (“You are the Robot”).

5–12 min — Direct Instruction / Model

Teacher says / does: Project the exact Minecraft click-path, model reading the NPC aloud, demonstrate movement controls, and show how to open MakeCode with C.

Students say / do: Watch the projection, repeat the controls aloud, and navigate to the lesson on their own device.

Differentiation / supports: A: desk controls chart and navigation-only first step. B: immersive reader pre-enabled. C: visible timer and explicit transition cues.

12–20 min — Guided Practice

Teacher says / does: Read the first NPC as a class, prompt “Stop, Read, Predict,” and model a first two-block movement sequence.

Students say / do: Read with a partner, predict the result before running, and try the first short sequence.

Differentiation / supports: A: teacher-initiated prediction routine. B: partner read-aloud and oral prediction accepted. C: pre-assigned Driver/Navigator roles.

20–38 min — Independent / Partner Work

Teacher says / does: Release students to reach the gold pressure plate and circulate using mini debug conferences: “What did you expect? What happened? What is one thing to try?”

Students say / do: Build, predict, run, and iterate. Finishers extend by adding one more block and testing again.

Differentiation / supports: A: three-step task card with checkpoints. B: partner continues to read NPC text and oral explanation is accepted. C: role cards remain visible and teacher checks in at 25 minutes.

38–45 min — Closure / Formative Assessment

Teacher says / does: Review major stumbling points and distribute Exit Ticket 1.

Students say / do: Share one success or stuck point with a partner, then complete the exit ticket.

Differentiation / supports: A: sentence-frame and checkbox version. B: drawing or dictation accepted. C: same closing format used every lesson.

Assessment: Teacher observation of navigation vs. conceptual fluency; prediction routine; Exit Ticket 1.

Tier 1 supports: controls anchor chart; NPC read-aloud routine; prediction protocol; partner support; sentence-frame exit ticket.

Tier 2 scaffolds: A — desk task card and teacher checkpoint before Run; B — immersive reader and partner read-aloud; C — pre-assigned Driver/Navigator roles and timer.

9.2 Lesson 2: Sequencing to Reach a Goal

Lesson Goal	Students will plan and write a longer sequence to move the Agent to a goal and explain their reasoning to a peer.
Objectives	Create a correct ordered sequence; use a planning tool before coding when needed; explain the algorithm using key vocabulary.
Materials	Minecraft Education, “Plan Before You Code” worksheet, visual sequence cards, vocabulary support card, Exit Ticket 2.

0–5 min — Engaging Opening

Teacher says / does: Display two short sequence cards and ask which one reaches the goal and why.

Students say / do: Study the map, discuss in pairs, and share a prediction with reasoning.

Differentiation / supports: A: colored path overlay. B: arrow-based visual cards. C: explicit 60-second decision prompt.

5–15 min — Direct Instruction / Model

Teacher says / does: Distribute the planning sheet and model sketching the Agent path, numbering steps, and translating the path into block code.

Students say / do: Complete a planning sheet for the new challenge and translate the path into code.

Differentiation / supports: A: planning sheet required before coding. B: drawing and oral description accepted. C: numbered boxes reduce open-endedness.

15–20 min — Guided Practice

Teacher says / does: Co-construct a path on the projected grid and ask at each step, “What goes next? Why?”

Students say / do: Offer block suggestions and write the agreed sequence onto the planning sheet.

Differentiation / supports: A: bounded turn called by name. B: may point instead of naming blocks. C: consensus-first structure reduces ambiguity.

20–38 min — Independent / Partner Work

Teacher says / does: Release Driver/Navigator pairs and confer around planning sheets. Offer a Repeat-block extension for finishers.

Students say / do: Code from the planning sheet, switch roles at the signal, and try the loop extension if ready.

Differentiation / supports: A: shortened path option and pre-Run check-in. B: bilingual vocabulary card and multimodal explanation accepted. C: explicit role boundary card remains posted.

38–45 min — Closure / Formative Assessment

Teacher says / does: Project one planning sheet and matching code, then ask whether the plan and code align. Distribute Exit Ticket 2.

Students say / do: Compare the plan to the code with a partner and complete the exit ticket.

Differentiation / supports: A: pre-printed sentence frames. B: drawing, demonstration, or oral response accepted. C: same exit-ticket format as Lesson 1.

Assessment: Planning sheet quality; sequence quality; vocabulary use; teacher observation notes.

Tier 1 supports: visible problem-solving routine; multiple task pathways; oral and written explanation prompts; predictable partner structure.

Tier 2 scaffolds: A — required planning sheet, shortened path, teacher check-in; B — bilingual vocabulary card and oral/gesture explanation accepted; C — boxed numbered planning sheet and role-boundary card.

9.3 Lesson 3: Debugging & Revision

Lesson Goal	Students will identify coding errors using the debug protocol, revise their algorithm, and explain what changed and why.
Objectives	Compare expected and actual outcomes; identify the error; apply the debug protocol; revise and retest; explain the fix.
Materials	Minecraft Education, debug protocol poster and desk card, tiered debug challenge sheets, Exit Ticket 3.

0–7 min — Engaging Opening

Teacher says / does: Project a broken code sequence and introduce the debug protocol: What did I expect? What actually happened? What is one thing to change? Try it. Repeat.

Students say / do: Inspect the code in pairs, identify the likely bug, and discuss possible fixes.

Differentiation / supports: A: printed protocol card and one-change rule. B: icon-based protocol card. C: same card and routine used previously.

7–20 min — Direct Instruction / Model

Teacher says / does: Intentionally break working code, think aloud through the protocol, and model staying with uncertainty rather than rushing to fix everything at once.

Students say / do: Suggest one fix, vote on the most likely correction, and observe the teacher test it.

Differentiation / supports: A: teacher names the self-regulation move explicitly. B: projected protocol mirrors desk card. C: each protocol step is named before it happens.

20–23 min — Guided Practice

Teacher says / does: Correct one shared code error together before independent work begins.

Students say / do: Suggest and vote on a fix, then watch the class run the corrected version.

Differentiation / supports: A: bounded turn at step 3. B: may point to a block rather than naming it. C: consensus-first structure reduces social ambiguity.

23–38 min — Independent / Partner Work

Teacher says / does: Distribute tiered debug challenges and assign levels using L1–L2 observations. Offer Expert Role to finishers with the rule that they may ask questions but not take over.

Students say / do: Work through the challenge using the protocol worksheet and record the bug and fix.

Differentiation / supports: A: Level 1 and teacher-assigned. B: icon-based worksheet and partner read-aloud available. C: Expert Role offered as a structured option.

38–45 min — Closure / Formative Assessment

Teacher says / does: Reframe debugging as the work of programming, invite two students to share a bug and fix, and distribute Exit Ticket 3.

Students say / do: Complete the exit ticket and name the protocol step used most.

Differentiation / supports: A: sentence frame plus checkbox. B: drawing or demonstration accepted. C: same closing format as previous lessons.

Assessment: Debug conference; code revision check; protocol worksheet; peer explanation.

Tier 1 supports: posted debug protocol; “one thing at a time” norm; whole-class model before independent work.

Tier 2 scaffolds: A — Level 1 challenge, desk protocol card, teacher-assigned level; B — icon-based protocol card and partner read-aloud; C — Level 1–2 challenge, Expert Role option, and identical exit-ticket structure.

9.4 Lesson 4: Efficient Solutions & Creative Build

Lesson Goal	Students will compare solution strategies using loops, complete a creative coding challenge with a loop constraint, and reflect on unit learning.
Objectives	Add a Repeat block to reduce code length; explain why a loop is more efficient; complete a creative build with at least one loop; reflect on unit learning.
Materials	Minecraft Education, loop comparison examples, planning sheet, Build Bridges world, Unit Reflection Exit Ticket 4.

0–7 min — Engaging Opening

Teacher says / does: Compare a 12-block movement sequence to a Repeat-4 loop and ask which is easier to write and debug.

Students say / do: Discuss the comparison with partners and explain why the loop is more efficient.

Differentiation / supports: A: physical blocks for counting iterations. B: visual comparison stays posted. C: explicit countdown to Chromebook transition.

7–20 min — Direct Instruction / Model

Teacher says / does: Model a Repeat block in MakeCode and introduce Build Bridges as a creative extension connected to later IGNITE projects.

Students say / do: Follow along with the first Repeat block and preview the extension world.

Differentiation / supports: A: three-step task card. B: narrated Minecraft library navigation. C: timer and clear release condition for extension work.

20–23 min — Guided Practice

Teacher says / does: Co-construct the first loop on the projector by asking how many repetitions are needed and which steps belong inside the loop.

Students say / do: Copy the loop onto the planning sheet and build it in MakeCode.

Differentiation / supports: A: teacher confirms the student plan directly. B: gesture response accepted. C: co-construction reduces loop-setup ambiguity.

23–38 min — Independent / Partner Work

Teacher says / does: Release students to either the loop challenge or Build Bridges with the requirement that at least one loop must be used. Document work through observation notes.

Students say / do: Code with the loop constraint, optionally use Driver/Navigator roles, and document the final result through a screenshot or paper plan.

Differentiation / supports: A: challenge option with completed plan sheet required. B: creative build option preferred if language load of the formal challenge is a barrier. C: teacher assigns the option and posts multiplayer rules.

38–45 min — Closure / Formative Assessment

Teacher says / does: Close by naming the growth from Lesson 1 to Lesson 4, post the reflection prompts, and distribute Exit Ticket 4.

Students say / do: Share one reflection aloud or in writing and complete the unit reflection exit ticket.

Differentiation / supports: A: teacher check-in before whole-class share. B: oral completion accepted. C: same structured closing routine used throughout the unit.

Assessment: Completion conference; loop use; Unit Reflection Exit Ticket 4.

Tier 1 supports: loop comparison activity; clear creative-work constraint; multiplayer option for engagement.

Tier 2 scaffolds: A — challenge option plus required plan sheet and teacher-assigned path; B — creative-build option and oral exit ticket; C — teacher-assigned option and identical exit-ticket format.

10 Reflection Prompts (Post-Implementation)

The following questions are grounded in the Warner lesson plan format and the ED452B inclusive design lens. Answer after implementation using observation notes, exit tickets, and student work samples.

Reflection Question	What to Look For	Design Implication for Next Lesson
Were instructions clear to all learners, including those who process text slowly or rely on visual/auditory input?	Were NPC instructions read aloud routinely? Did any student get stuck because they could not access the text?	If students skipped NPC text and got stuck → pre-print NPC instructions on physical card. If immersive reader unavailable → pre-set before session.
Were students engaged? Which students showed disengagement and what was the likely root cause?	Distinguish: boredom (extension path insufficient) vs. barrier (stuck, overwhelmed) vs. mismatch (task too open-ended).	High-performers disengaged → formalize Expert Role and loop extension earlier. Stuck students shut down → add stuck card protocol explicitly at lesson start.
Could all students participate meaningfully — including Students A, B, and C?	Did Student A use the task card? Did Student B explain code orally? Did Student C benefit from Driver/Navigator role?	Any identified student who could not participate due to access barrier → elevate to Tier 2 check-in next session. Report pattern to cooperating teacher.
Was the lesson objective met? What is the evidence?	Exit tickets: can students name what their Agent did and identify a change? Planning sheets match code? Debug protocol used rather than random clicking?	Most students could not explain code → more oral modeling in next session. Debug protocol not used → revisit explicitly before L3 independent challenge.
How do student responses inform the design of the next lesson?	What stumbling blocks appeared (pilot data: loading times, coordinates, NPC skipping, device issues)? What extensions were needed?	Adjust Tier 1 and Tier 2 supports based on exit ticket patterns. Identify students for Expert Role in L3–4. Flag students needing pre-teaching.

11 Unit Analysis

11.1 Analysis of Student Learning

Student learning in this unit is analyzed through patterns across the four Minecraft lessons rather than through completion alone. The central question is not simply whether students finished the coding challenges, but whether they demonstrated computational thinking once access barriers such as sign-in, world entry, NPC reading, task interpretation, and interface navigation were separated from the coding concepts themselves. The teacher should therefore examine whether students increasingly demonstrate independence with algorithms, explain how their code works, debug more strategically over time, and use supports such as planning sheets, partner roles, and click-path charts without lowering expectations.

- Loading times (2–10 min) and device issues are administrative variables, not cognitive ones — address at setup, not mid-lesson.
- Students who beat Lesson 1 quickly (Kade, Lorenzo, Hadi, Hani) showed they can self-extend; Expert Role and loop extension formalize this.
- NPC skipping was the single largest access barrier in the pilot — every Tier 1 support in this unit addresses it directly.
- Coordinate confusion in the fill-with-air command confirms that spatial vocabulary must be pre-taught before students encounter it independently.
- Peer-to-peer help (“Did you ask [name] since they already beat this part?”) occurred naturally in the pilot — Driver/Navigator formalizes and extends this existing student norm.

11.2 Observed Learning Patterns

Setup and interface friction affected what learning was visible. Students needed support with sign-in, menu navigation, finding the correct world, and entering the correct coding environment before their understanding of sequencing or debugging could be assessed. Early difficulty could not automatically be interpreted as weak computational thinking because some students were stuck before the coding task truly began. For that reason, the final assessment design tracks startup independence separately from code accuracy.

Students needed the task goal made visible. In the timed dual-plate challenge, several students did not initially understand that both the student and the Agent needed to stand on separate gold plates at the same time. This misunderstanding affected performance even when students had the technical ability to build movement code. The unit responds with visible success criteria, teacher think-alouds, and the Stop–Read–Predict routine.

Debugging became stronger when it was procedural. When students were simply told to try again, some repeated the same mistake or added blocks randomly. When the teacher or peer helper asked students to count blocks, predict the Agent’s route, test the code, and revise one part at a time, debugging became observable learning. This supports assessing bug explanation and revision evidence, not only final task completion.

Collaboration needed structure. Open-ended partner work can support learning, but it can also allow one student to take over while another disengages. The Driver/Navigator structure preserves peer support while maintaining individual accountability. Expert Helper roles also allow students with prior Minecraft knowledge to support classmates by asking questions rather than taking over

the keyboard.

Explanation lagged behind completion. Some students could complete parts of the challenge but struggled to explain the underlying code choices. This matters because the standards call for algorithmic reasoning, not only successful navigation. The unit therefore includes oral conferences, exit-ticket sentence frames, and final reflection prompts so students practice explaining what their Agent did, what bug occurred, and why one solution was more efficient.

Evidence Pattern	Interpretation	Instructional Adjustment
Many students needed help entering the correct Minecraft lesson or reading directions.	Startup and reading load were access barriers, not direct evidence of weak CS thinking.	Keep click-path projection, NPC read-aloud, and stop-and-check slide as Tier 1.
Students misunderstood the dual-plate goal.	Students needed success criteria clarified before coding.	Add task-goal restatement before Run and use a visual success checklist.
Students solved parts of the path after hints about counting, floating, or route efficiency.	Specific hints made computational reasoning visible without giving the answer away.	Use hints-before-rescue protocol and document which hint moved the student forward.
Students varied in ability to explain code after completing it.	Completion and explanation are separate outcomes.	Use oral conferences, sentence frames, and partner teach-back.
Some advanced students moved ahead quickly.	Prior Minecraft expertise is an instructional asset.	Use Expert Helper roles and loop/shortcut extensions.

11.3 Unit Strengths

The strength of this unit is that it preserves the authentic Minecraft coding sequence while organizing it into explicit instructional design. Students engage in real coding tasks, but the unit does not assume that motivation alone will produce access. Instead, it pairs a high-interest platform with clear routines, support structures, and layered assessment. For an inclusive classroom, that combination is more instructionally responsible than assigning the digital task without redesign.

11.4 Problems, Dilemmas, and Next Revision

The main challenge is that Minecraft introduces several access barriers at once. Some students are slowed by login and loading time. Others skip or misread NPC directions. Some students understand movement logic but struggle to translate it into a long enough command sequence. A few students move through the game quickly but still need support explaining why their solution works. These patterns challenge any assessment system based only on task completion.

A second dilemma is the balance between productive struggle and rescue. In a fast-paced classroom, it is tempting for the teacher or peer helper to fix the code quickly so the student can move on.

However, this can hide whether the student actually understood the bug. The final unit addresses this by using a hints-before-rescue protocol and requiring students to explain at least one change.

In the next revision, I would keep the Minecraft context but tighten the entry routines. I would begin each session with a projected click path, a one-minute review of the task goal, and a visible success checklist. I would also keep the Driver/Navigator roles but make role-switching more explicit so that both partners explain the code. I would collect a stronger set of artifacts: screenshots of code, planning sheets, debugging notes, exit tickets, and short teacher-conference notes. This would make the final analysis stronger and provide better evidence for students with IEP-related needs.

11.5 Target Student Assessment and Future Instructional Decisions

Student A. Student A's likely learning process is characterized by strong idea generation but inconsistent self-monitoring. If Student A completes a path only after repeated teacher redirection, the data would suggest the need for stronger executive-function scaffolding rather than lower coding expectations. Future instruction should keep the planning sheet and teacher checkpoint in place until Student A can independently state the goal, identify the first three code blocks, and predict the outcome before pressing Run.

Student B. Student B's assessment should prioritize whether the student can demonstrate and explain the algorithm through oral, visual, or partner-supported modes. If written exit tickets are weak but oral explanation and code behavior are strong, the instructional decision should be to maintain language scaffolds rather than reduce the computational demand. Future instruction should include vocabulary preview, visual examples, and oral explanation as valid evidence.

Student C. Student C's learning outcomes should be interpreted in light of regulation and role clarity. If Student C participates successfully when roles are explicit but withdraws during open-ended choice, the data indicate a need for predictable collaboration structures. Future teaching should preserve the Driver/Navigator routine, provide a bounded option set, and use the stuck-card system before frustration escalates.

11.6 Collaboration

This unit reflects collaboration among the inclusive educator, the cooperating teacher, students, and existing curriculum resources. The cooperating teacher's implementation of Minecraft lessons provided the real classroom evidence that shaped the redesign. The classroom transcripts and challenge booklet function as professional collaboration artifacts because they show what students actually did, where they became stuck, and which teaching moves helped. Students also contributed to the instructional design by revealing which supports worked: peer help, specific hints, goal restatement, and visible routines.

In future iterations, collaboration should become even more systematic. Before teaching the unit, I would meet with the cooperating teacher to review the target-student supports, assign partner roles, and decide what evidence will be collected. After each lesson, I would debrief quickly: Which students needed Tier 2 support? Which supports helped? Which goal or direction caused confusion? This would align the unit more fully with CEC collaboration expectations and make the evidence trail stronger for student learning analysis.

12 References (APA 7th Edition)

- CAST. (2018).* Universal Design for Learning guidelines version 2.2.
<http://udlguidelines.cast.org>
- Karten, T. J. (2021).* Inclusion strategies that work: Research-based methods for the classroom (4th ed.). Corwin.
- Margolis, J., Estrella, R., Goode, J., Holme, J. J., & Nao, K. (2017).* Stuck in the shallow end: Education, race, and computing (Updated ed.). MIT Press.
- Microsoft Education. (2023).* Minecraft Education: Computer science curriculum.
<https://education.minecraft.net/en-us/resources/computer-science-subject-kit>
- Paris, D., & Alim, H. S. (Eds.). (2017).* Culturally sustaining pedagogies: Teaching and learning for justice in a changing world. Teachers College Press.
- New York State Education Department. (2020).* New York State K-12 computer science and digital fluency learning standards. <http://www.nysed.gov/curriculum-instruction/computer-science-and-digital-fluency-learning-standards>
- Computer Science Teachers Association. (2017).* CSTA K-12 CS standards.
<https://www.csteachers.org/page/standards>
- International Society for Technology in Education. (2016).* ISTE standards for students.
<https://www.iste.org/standards/iste-standards-for-students>
- Schirmer, E. (2026).* ED452B Instructional strategies for inclusive classrooms B — syllabus and rubric. University of Rochester, Warner School of Education.